

CP-Bench: Evaluating Large Language Models for Constraint Modelling

Kostis Michailidis^{a,*}, Dimos Tsouros^a and Tias Guns^a

^aDTAI, KU Leuven, Belgium

Abstract. Combinatorial problems are present in a wide range of industries. Constraint Programming (CP) is a well-suited problem-solving paradigm, but its core process, namely constraint modelling, is a bottleneck for wider adoption. Aiming to alleviate this bottleneck, recent studies have explored using Large Language Models (LLMs) as modelling assistants, transforming combinatorial problem descriptions to executable constraint models, similar to coding assistants. However, the existing evaluation datasets for constraint modelling are often limited to small, homogeneous, or domain-specific instances, which do not capture the diversity of real-world scenarios. This work addresses this gap by introducing **CP-Bench**, a novel benchmark dataset that includes a diverse set of well-known combinatorial problem classes sourced from the CP community, structured explicitly for evaluating LLM-driven CP modelling. With this dataset, and given the variety of constraint modelling frameworks, we compare and evaluate the modelling capabilities of LLMs for three distinct constraint modelling systems, which vary in abstraction level and underlying syntax: the high-level MiniZinc language and Python-based CPMpy library, and the lower-level Python interface of the OR-Tools CP-SAT solver. In order to enhance the ability of LLMs to produce valid constraint models, we systematically evaluate the use of prompt-based and inference-time compute methods adapted from existing LLM-based code generation research. Our results underscore the modelling convenience provided by Python-based frameworks, as well as the effectiveness of documentation-rich system prompts, which, augmented with repeated sampling and self-verification, achieve further improvements, reaching up to 70% accuracy on this new, highly challenging benchmark.

1 Introduction

Combinatorial problems exist in numerous real-world applications, where optimal decision-making is essential, including logistics, scheduling, and network design [24]. Solving these problems requires finding the best combination of elements under specific constraints, which is computationally intensive; many such problems are NP-hard, meaning that their solving time can increase exponentially with problem size in the worst case.

Constraint Programming (CP) is an established paradigm for solving combinatorial problems, employing a declarative approach in which users specify *what* constraints the solution must satisfy, rather than detailing *how* to find it [35, 30]. Despite its solving effectiveness, the process of modelling—translating a problem description into a formal CP model—remains a significant bottleneck, as it is

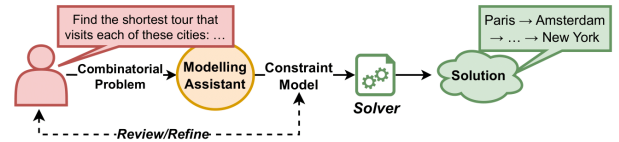


Figure 1: LLM-driven constraint modelling: users state a combinatorial problem in natural language, which the system transforms into a formal constraint representation, and delegates the latter to a constraint solver.

inherently complex. It involves identifying and formally defining the problem’s decision variables with their domains, constraints on the variables, and, where applicable, an objective function. This process demands a deep understanding of the application domain and proficiency in the chosen modelling framework, both syntactically and semantically. This modelling complexity is recognised as one of the most important limitations for the wider use of combinatorial optimisation techniques, restricting accessibility for non-experts [9, 8].

This raises a central question: Can we lower the modelling barrier to make CP accessible to a broader audience? In this direction, recent AI advances in large language models (LLMs) offer the opportunity of creating *modelling assistants* that could initiate or (semi-)automate the modelling process (as illustrated in Figure 1), similarly to how LLM-based coding assistants are widely being used for a variety of code-related tasks [15, 4, 7].

Nevertheless, LLM-driven constraint modelling presents different challenges compared to imperative code generation. First, while imperative code specifies step-by-step sequential instructions, declarative modelling requires that all defined properties and constraints must hold true simultaneously. Second, formalizing the extracted CP components—decision variables, domains, constraints, and objective function—is hard due to the combinatorial nature of modelling choices themselves. Specifically, modelling requires selecting a *viewpoint*—a specific way to represent variables and structure constraints based on the problem domain—from potentially many alternatives [10]. Third, verifying the correctness of a generated constraint model is different from imperative code generation, in which correctness can often be tested using example inputs and expected outputs (e.g., unit tests). In contrast, constraint modelling lacks such verifications, as a “test” would require knowing the correct model or solution in advance.

Recent works have explored the potential of LLMs on this task by generating intermediate solvable representations of numerical reasoning problems in formal declarative modelling frameworks

* Corresponding Author. Email: kostis.michailidis@kuleuven.be.

[38, 33]. Indeed, promising results have been achieved for simple domain-specific problems, such as small-scale linear optimization problems [29, 27] and logic grid puzzles [13], but scaling to more complex and diverse problems remains a significant challenge [20].

Existing limitations in the field can be summarized as follows. (a) Current NL-to-CP benchmarks lack the diversity and complexity necessary to represent practical combinatorial problems. This constitutes a prerequisite for assessing LLM capabilities on this task. (b) Given the wide variety of constraint modelling frameworks available, ranging in abstraction levels and interface types, there is a need for systematic evaluation across them to explore their impact on LLMs. (c) Applying recent LLM advances to combinatorial modelling remains underexplored, even though test-time scaling methods have shown success for complex programming tasks [5].

To address the lack of representative benchmarks, we have curated CP-Bench, a novel dataset of diverse combinatorial problems, which includes self-contained descriptions of problems gathered from distinct sources within the CP community. While existing repositories (e.g., PyCSP3 examples [17], CSPLib [11], MiniZinc benchmarks suite [22], and others) contain models, they are not structured for automated evaluation of LLM-generated models. We designed CP-Bench for this task based on solution-level evaluation, which is non-trivial since more than one (optimal) solution may exist for a combinatorial problem (detailed in Section 5.2).

Using this benchmark, we systematically evaluate the capabilities of state-of-the-art LLMs when prompted to generate models using different frameworks, varying by abstraction level (low vs. high) and interface type (domain-specific vs. Python-based). We select the following as representatives for each category: OR-Tools [26, 25] offers a low-level, direct solver Python API; CPMpy [12] provides a high-level, Python-based modelling interface; and MiniZinc [22] is a well-established high-level, domain-specific language for constraint modelling. Across these frameworks, we compare different system prompts, starting from basic minimal instructions and progressing to detailed guidelines and including documentation specific to the framework used.

With inference-time computation showing high success rates across various LLM tasks [32], we adapt and evaluate such methods in order to further enhance the quality of the generated constraint models. Firstly, we use Retrieval-Augmented In-Context Learning (RAICL) to enrich the prompt context with representative examples [39, 20]. Secondly, we exploit the probabilistic nature of LLMs, by repeatedly sampling multiple CP models and employing solution majority voting to select the most reliable model based on the observed outputs [4, 23]. Finally, to address detectable errors, such as runtime or solution printing ones, and trigger self-correction of modelling errors, we investigate the effect of self-verification prompting on LLMs for iterative model refinement.

The primary contributions of this work are the following:

- A novel benchmark comprising 101 diverse combinatorial problem descriptions along with their runnable CP models, structured specifically for evaluating LLM-driven constraint modelling.
- Systematic evaluation of state-of-the-art LLMs across three commonly used constraint modelling frameworks, demonstrating the impact of framework interface levels on modelling accuracy, with Python-based frameworks achieving up to 65% accuracy compared to a maximum of 50% for the domain-specific language.
- Adaptation of prompt-based and inference-time compute methods for enhancing LLM-driven constraint modelling, reaching up to 70% accuracy.

2 Background

2.1 Constraint Programming

We use CP as a formal intermediate representation for the given combinatorial problems. A *Constraint Optimization Problem* (COP) is a type of combinatorial problem that involves finding an optimal assignment of values to decision variables, subject to a set of constraints and an objective function. Formally, a COP is defined as a tuple $(\mathbf{X}, \mathbf{D}, \mathbf{C}, f)$ where:

- $\mathbf{X} = \{x_1, x_2, \dots, x_n\}$ is a set of n decision variables.
- $\mathbf{D} = \{D_1, D_2, \dots, D_n\}$ is a set of n domains, where each D_i is a finite set of allowable values for x_i .
- $\mathbf{C} = \{C_1, C_2, \dots, C_m\}$ is a set of m constraints, with each C_j specifying allowed combinations of values for a subset of variables. Formally, $C_j \subseteq D_{j1} \times D_{j2} \times \dots \times D_{jk}$ for some subset $\{x_{j1}, x_{j2}, \dots, x_{jk}\} \subseteq \mathbf{X}$.
- $f : \prod_{i=1}^n D_i \rightarrow \mathbb{R}$ is the objective function to be optimized (either maximized or minimized).

The goal is to find an assignment $\mathbf{a} = \{x_1 = v_1, x_2 = v_2, \dots, x_n = v_n\}$ that satisfies all constraints in $\mathbf{C}$ and optimizes the objective function f . Formally, an assignment $\mathbf{a}$ is feasible if $v_i \in D_i$ for all i and if $(v_{j1}, v_{j2}, \dots, v_{jk}) \in C_j$ for each constraint C_j . An optimal solution $\mathbf{a}^*$ is a feasible assignment that optimizes f . If no objective function f is defined, the problem reduces to a *Constraint Satisfaction Problem* (CSP), where the goal is to find any feasible assignment without optimization criteria. When no feasible assignment exists, the problem is considered unsatisfiable. For brevity, we will use $\mathbf{a}^*$ to represent either an optimal solution in COPs or a feasible solution in CSPs, and we will commonly refer to it as the *optimal solution*.

Notably, CP systems are not restricted to linear constraints but also support a variety of global constraints that capture common patterns in combinatorial problems. These include min/max aggregators, element constraints (for array access), count/nr-values (for aggregation over arrays), cumulative constraints (for scheduling-type problems), and many others that facilitate more expressive modelling of complex problems [11].

2.2 Large Language Models

Based on the Transformer architecture [34], LLMs are deep learning systems, typically with billions of parameters, capable of learning and generating complex language patterns [18]. In this work, we use them as black-box text generators, configured only by an input text (or prompt) and a randomness parameter (temperature). Formally:

$$LLM(p, \tau) = (x_1, x_2, \dots, x_w), \quad \text{where} \quad (1)$$

$$x_{t+1} \sim P_\tau(x_{t+1} \mid p, x_{\leq t})$$

Here, $x_{\leq t}$ denotes the sequence of tokens generated up to time step t , with $x_{\leq 0}$ as the empty sequence and x_1 as the first generated token. At each step, the next token x_{t+1} is sampled from the temperature-adjusted probability distribution $P_\tau(x_{t+1} \mid p, x_{\leq t})$, which is typically computed by applying the softmax function to the model's logits—raw unnormalized scores over the vocabulary. The temperature τ scales the logits before softmax, with higher values increasing randomness and diversity, while lower values favour the most likely tokens. The generation process continues until a predefined stop token

is encountered or the maximum context length is reached. We assume tokenization and detokenization into text are handled internally by the LLM.

3 Problem Formulation

As illustrated in Figure 1, we conceptualize a system capable of assisting users in modeling combinatorial problems, transforming their textual descriptions into executable models that a constraint solver can solve. In this paper, we focus on exactly this transformation, which requires a mechanism that can process and accurately represent the problem’s constraints, variables, and objectives—tasks that demand expert knowledge in both the application domain and CP modelling. We consider LLMs such versatile tools, because of their powerful natural language processing and code-synthesizing capabilities [4].

Formally, we define the process as follows: Let $\mathcal{V}^*$ represent the set of all possible text sequences that can be composed using tokens from the LLM’s vocabulary $\mathcal{V}$. Let $\mathcal{M}$ be the space of all possible constraint models, and let $\mathcal{S}$ be the space of all possible valid assignments (solutions) to the decision variables of a constraint model. We utilize the previously defined function LLM , which maps an input prompt $p \in \mathcal{V}^*$ and a temperature parameter $\tau \in \mathbb{R}$ to an output text that contains a constraint model $m \in \mathcal{M}$. Next, we define a function $Solver : \mathcal{M} \rightarrow \mathcal{S} \cup \{\emptyset, error\}$. Given a constraint model m , the Solver yields an output $\mathbf{a}^+$, which represents either an optimal solution $\mathbf{a}^* \in \mathcal{S}$, an unsatisfiable model $\emptyset$, or some kind of solver error (e.g., syntax or runtime). The overall system can be represented as follows:

$$Solver(LLM(p, \tau)) = \mathbf{a}^+ \quad (2)$$

4 Methodology

The task for LLMs in this context is to translate problem descriptions into a declarative specification of variables and constraints over them, including expressing them correctly in a specific modelling framework and generating code for printing the computed solution in a structured way. These latter requirements demand coding skills; thus, we take inspiration from successful test-time scaling techniques used for generating imperative code.

4.1 System Prompt

Given the complexity of translating natural language descriptions into constraint models, we explore different levels of system prompts to guide LLMs more effectively in a 0-shot setting. Without examples in the prompt (hence 0-shot), providing a comprehensive description of the task is important to help the LLM process and correctly model the problem declaratively. Formally, if $sys \in \mathcal{V}^*$ is the system prompt describing the task, and $c \in \mathcal{V}^*$ is the combinatorial problem description, then the LLM input prompt is $p = sys \oplus c$, concatenating the system prompt and the problem description. We examine three levels of system prompts, each progressively providing more detailed information (the exact prompts used per framework can be found in the Appendix):

1. **Basic prompt.** The basic prompt provides essential instructions to the LLM, only describing the task of producing a constraint model using a specific framework and how the solution should be

formatted. This prompt simulates a scenario where a user gives minimal guidance, e.g. as one would do in a chat interface.

2. **Guidelines prompt.** Expanding on the basic prompt, this level adds specific guidelines related to the modelling process. These guidelines include code generation steps, generic classroom-style advice on constraint modelling, and a template on how to format the response model in the modelling framework used. It mirrors a situation where a user provides more elaborate but still generic instructions.
3. **Documentation prompt.** The most detailed prompt builds on top of the guidelines prompt, appending single-line API documentation of the available classes and functions of the CP modelling framework. This prompt offers (limited) access to solver documentation, enabling direct reference to available methods and functionalities.

4.2 In-Context Learning

Few-shot prompting is a widely used method that includes task-relevant examples in the input prompt, improving LLM responses without requiring retraining [2]. However, selecting effective examples is critical, as a static or random set of examples may not align well with diverse inputs [19, 37]. To address this, recent studies have examined dynamically inserting examples in the prompt context based on the current input [39]. In the context of constraint modelling, adding examples of semantically similar problems and their corresponding models has previously improved the accuracy of generated models [20].

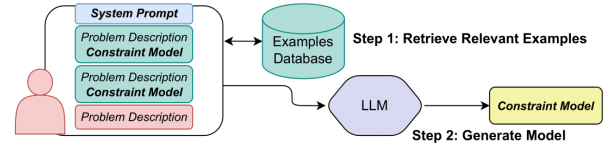


Figure 2: Retrieval-augmented few-shot prompting: on-the-fly example retrieval to provide more relevant patterns to the LLM.

Formally, this method retrieves from a prebuilt database an ordered set E of e input-output pairs, $E = \{(i_j, o_j)\}_{j=1}^e$, based on their relation to the given problem c . Given a database of problem-model pairs, the retrieval process involves comparing the token embeddings of c with each problem in the database and selecting the e most similar ones using a predefined metric, such as semantic similarity [3]. The prompt then becomes $p = sys \oplus E \oplus c$, where the retrieved examples E are placed between the system prompt and the current problem description, as shown in Figure 2.

4.3 Repeated Sampling

While enriching the prompt with instructions and examples can enhance the guidance provided to LLMs, it does not fully address the complexity of synthesizing CP models from textual descriptions.

In this direction, we aim to benefit from the probabilistic nature of LLMs through repeated sampling [4]. Instead of relying on the single “most probable” output, we increase the temperature parameter value τ , generating multiple candidate models for the same problem description. This strategy exploits the variability in LLM outputs to identify the most consistent model across multiple runs.

Specifically, we apply *Solution Majority Voting* for the final selection of the generated model [23]. By repeatedly querying the LLM

¹ We assume that input p includes both a problem description and instructions for a specific modelling framework.

Algorithm 1 Sampling with Solution Majority Voting

Require: (p, k, τ) // Prompt, #Samples, Temperature
1: $\mathcal{C}, \mathcal{S} \leftarrow \emptyset, \emptyset$ // Candidate CP models & solutions
2: **for** $i = 1$ to k **do**
3: $m_i \leftarrow \text{LLM}(p, \tau)$ // Generate CP model
4: $s_i \leftarrow \text{Solver}(m_i)$ // Extract the solution
5: $\mathcal{C}, \mathcal{S} \leftarrow \mathcal{C} \cup \{m_i\}, \mathcal{S} \cup \{s_i\}$
6: **end for**
7: $s_{\text{maj}} \leftarrow \text{most_freq}(\mathcal{S})$ // Find most frequent solution
8: **if** s_{maj} exists **then**
9: **return** $\mathcal{C}[\text{first_index_of}(s_{\text{maj}}, \mathcal{S})]$
10: **end if**
11: **return** $\mathcal{C}[1]$ // Fallback to first candidate model

and solving each generated constraint model, we observe the solutions they produce; then, the first model yielding the most frequently observed solution is selected. If none of the models produces a solution, we default to the first generated model. The process is outlined in Algorithm 1.

4.4 Self-Verification

Writing semantically and syntactically correct code in a single pass, without the ability to revise previous steps, is a complex task even for experienced programmers. This challenge holds even more for CP modelling, where variables need to be declared before constraints, and constraints should not contradict earlier ones.

We expect that the generated models may occasionally contain runtime, modelling or formatting errors. This motivates us to define three criteria that assert the validity of the generated constraint model:

- **Runtime Integrity:** The generated code executes without errors and correctly utilizes library components.
- **Model Accuracy:** The decision variables, constraints, and objective function are correctly defined and aligned with the problem description; when the model is executed, the produced solution is valid and optimal.
- **Solution Output:** The solution is printed in the correct structured format as per the user’s provided instructions.

A promising approach to improve the reliability of code generation is to allow LLMs to review, verify, or refine their previous output iteratively, effectively mimicking the human code debugging process [5, 36].

We composed a *Self-Verification* base prompt v that guides the LLM through an evaluation and correction process, instructing it to iteratively assess the model based on the aforementioned criteria. Appended to this prompt are: (a) the system prompt sys , (b) the problem description c , (c) the model m generated by the LLM in the previous attempt, and (d) its resulting output a^+ when executed. Formally, the full self-verification prompt at iteration t is denoted as $v \oplus sys \oplus c \oplus m_{t-1} \oplus a_{t-1}^+$. This process continues until either the LLM determines that the most recent constraint model m_{t-1} is correct according to the specified criteria, or a predefined number of iterations is reached; at that point, the latest model is selected. The process is depicted in Figure 3.

5 Dataset: CP-Bench

The evaluation of LLMs on translating textual problem descriptions into formal CP specifications has so far been limited by the scope and

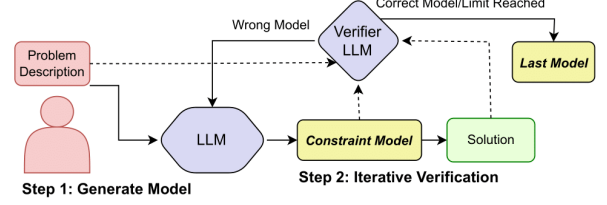


Figure 3: Iterative Self-Verification: the generated CP model is verified iteratively; both the problem and produced solution are also provided back to the LLM. In this work, we use a single LLM for both model generation and verification (thus, self-verification).

diversity of available datasets. Existing benchmarks, such as NL4Opt [28]—part of a competition at NeurIPS²—and *Logic Grid Puzzles* (LGPs) [14], primarily feature small, domain-specific problems, such as simple linear programming (LP) problems or homogeneous puzzles, as summarized in Table 1. More recently, Singirikonda et al. [31] presented a cross-domain dataset with MiniZinc models, focusing mostly on LP and Mixed-Integer Programming (MIP) problems.

To address the gap in CP-modelling LLM benchmarks [20], we introduce **CP-Bench**³ [21], a collection of 101 diverse and challenging combinatorial problems drawn from well-established sources in the CP community, as detailed in Table 2. To achieve diversity, we selected problems from the popular “benchmark library for constraints” CSPLib⁴ [11], from the online examples repository of the CPMpy modelling framework⁵ [12], from the extensive online repository of Håkan Kjellerstrand⁶ [16], and from a modelling course-based set of problems⁷ [20].

Table 1 highlights the diversity and complexity of CP-bench compared to existing benchmarks. It spans optimization and satisfaction problems, with a wide range of decision variables, constraints, and 241 distinct constraint types. In particular, the number of constraints per problem ranges from 1 to 2017, and the number of variables from 3 to 962, offering a significantly broader scope than NL4Opt and LGPs (Table 1).

5.1 Instance Structure

Each dataset instance was transformed into a self-contained Python file containing all the necessary components for evaluating the modelling capabilities of LLMs, with the following structure:

1. **Metadata:** The source(s) of the problem along with any additional relevant information.
2. **Problem Description:** The original problem statement as provided by the source, with an additional print instruction that specifies the required output structure.
3. **Input Data (optional):** For parameterized problems, one valid data instance is provided in plain Python syntax, using only built-in types.

² <https://neurips.cc/virtual/2022/competition/50079>

³ The dataset is available at <https://huggingface.co/datasets/kostis-init/CP-Bench>

⁴ Accessed on May 23, 2024: <https://www.csplib.org/>

⁵ Accessed on May 24, 2024: <https://github.com/CPMpy/cpm.py/tree/master/examples>

⁶ Accessed on May 26, 2024: <http://www.hakank.org/cpm.py/>

⁷ Accessed on July 7, 2024: https://github.com/kostis-init/CP-LLMs-ICL/tree/main/data/APLAI_course

Table 1: Comparison of CP-Bench with existing datasets for evaluating LLM-generated CP models of combinatorial problems. Unique constraints count distinct constraint operators and their argument structures (arity and type, e.g., variable/constant/expression).

Benchmark	#	Constraints		Decision Variables		# Unique Constraints	# Opt. Problems
		Average	Min/Max	Average	Min/Max		
NL4Opt	289	2.90 $\pm$ 0.81	2/5	2.02 $\pm$ 0.13	2/3	27	All
LGP	100	38.47 $\pm$ 15.60	7/69	12.00 $\pm$ 0.00	12/12	8	None
CP-Bench [ours]	101	84.76 $\pm$ 274.90	1/2017	51.69 $\pm$ 125.03	3/962	241	30

Table 2: CP-Bench sources and selected instances. Selection was based on CPMpy model availability to facilitate evaluation, for which the ground-truth model is necessary.

Source	#	Selected Problems
CSPLib ^[4]	25	IDs: 1, 2, 5–13, 15, 19, 21, 24, 26, 28, 33, 44, 49, 50, 53, 54, 76, 84.
CPMpy Examples ^[5]	16	<i>bus_scheduling</i> , <i>minesweeper</i> , <i>packing_rectangles</i> , <i>jobshop</i> , <i>knapsack</i> , <i>mario</i> , <i>n_puzzle</i> , <i>RCPS</i> , <i>room</i> , <i>send</i> , <i>set_game</i> , <i>sudoku</i> , <i>tsp</i> , <i>agatha</i> , <i>wolf</i> , <i>zebra</i> .
Håkan K. ^[6]	42	From <i>3_coins</i> to <i>crypta</i> (alphabetically), excluding test files.
Course ^[7]	18	All problems from the repository.

- Model:** The decision variables, constraints, and objectives (if any) that make up a runnable CP model of the problem, using the CPMpy modelling framework.
- Solution Printing:** Solution printing code, following the requirements in the problem description.

Importantly, only the **Problem Description** and (optionally) **Input Data** are provided to the LLM, the remaining components are used exclusively for evaluation.

5.2 Evaluation Process

As the focus lies on problems that may have more than one optimal solution, it is mandatory to adopt an appropriate metric to evaluate the generated constraint models. Related works have used constraint-level, model-level, and solution-level accuracy [28, 20]. The first two metrics require mapping the decision variables of the generated model with those from the ground-truth model. However, when modelling complex problems, multiple possible viewpoints and formulations of the decision variables are possible, thus making it highly challenging to map variables between the generated and ground-truth models in a correct but generic way. This leads us to opt for solution accuracy, which measures the correctness of the solution obtained by executing the generated code; correctness is evaluated with respect to the ground-truth model, considering both feasibility and optimality. Additionally, this metric allows for evaluating LLMs on **any** modelling framework desired, as long as a final solution can be printed.

Formally, let N be the total number of test instances, $\hat{\mathbf{a}}_i$ the solution produced by the generated CP model of instance i , and $\mathbf{a}_i^*$ an optimal solution from the ground truth constraint model $\mathbf{C}_i$. Solution accuracy is defined as:

$$\text{SA}(\hat{\mathbf{a}}) = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(\hat{\mathbf{a}}_i \in \text{Sol}(\mathbf{C}_i) \wedge f_i(\hat{\mathbf{a}}_i) = f_i(\mathbf{a}_i^*)) \quad (3)$$

Here, the term $\hat{\mathbf{a}}_i \in \text{Sol}(\mathbf{C}_i)$ tests whether the generated solution satisfies all constraints of the ground-truth model. For optimization problems, the second condition $f_i(\hat{\mathbf{a}}_i) = f_i(\mathbf{a}_i^*)$ ensures equivalence to the optimal objective value.

6 Experiments

In our experiments, we focus on the following questions:

- Q1:** How do LLMs perform in generating constraint models across different modelling frameworks?
- Q2:** What types of errors do LLMs make when generating constraint models, and how do different system prompts address them?
- Q3:** What is the impact of inference-time compute methods, such as few-shot prompting, repeated sampling, and self-verification, individually and in combination?

6.1 Experimental Setup

Table 3 lists the LLMs used throughout the experiments. We considered covering a range of properties (coding, reasoning, rank, mixture of experts, number of parameters, open weights/proprietary, context window size, etc.). Specifically, we selected 8 models from the LMSys Chatbot Arena leaderboard focused on coding tasks [6]^[8]. Notably, gpt-4.1-mini was preferred to gpt-4.1 (it performed almost equivalently), and gpt-4.1-nano (it showed very low performance). We utilized the OpenAI^[9], Together AI^[10], Anthropic^[11], and DeepSeek^[12] APIs via their Python client libraries, using Python 3.12. For executing the generated constraint models, we used a 10-second timeout. Finally, we set the temperature parameter τ to 0 for deterministic outputs, except when applying repeated sampling techniques, where we increased it to $\tau = 0.8$ for variability, following existing works [23].

6.2 Results & Discussion

We now present the results and discuss them.

6.2.1 Q1: Modelling Framework

Figure 4 shows the detailed results, and Figure 5 illustrates, aggregated across all LLMs, the average percentage of successfully generated models for each modelling framework (MiniZinc, CPMpy, and the OR-Tools CP-SAT solver) for the different system prompt levels.

The results indicate that the choice of modelling framework influences the success rates of LLMs: when instructed to utilize the

⁸ As accessed on 04 May 2025.

⁹ <https://platform.openai.com/docs/api-reference>

¹⁰ <https://docs.together.ai/reference/chat-completions>

¹¹ <https://docs.anthropic.com/en/api/getting-started>

¹² <https://platform.deepseek.com/docs>

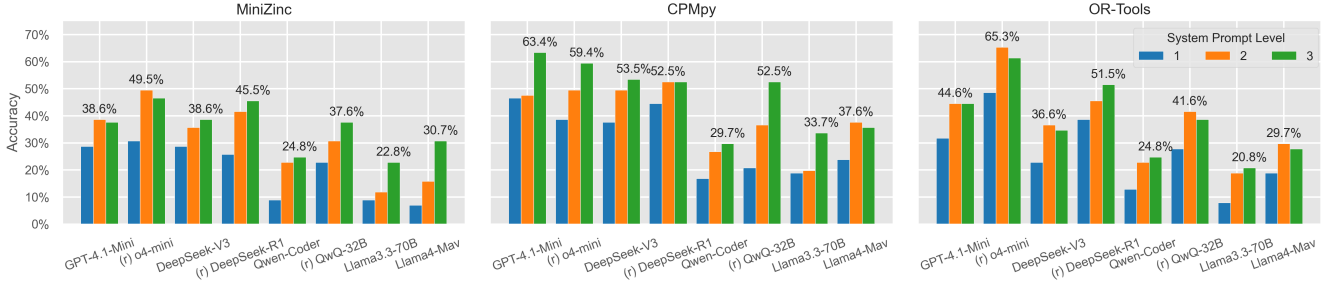


Figure 4: Percentages of successfully generated models. From left to right: MiniZinc, CPMpy, OR-Tools. Reasoning LLMs are prepended with (r).

Table 3: LLMs used in the experiments. Parameter counts and context sizes are approximate, ‘*’ denotes active parameters for Mixture-of-Experts models, and N/A means we found no public information about it.

LLM	Org.	# Params	Context	Reasoning
gpt-4.1-mini-2025-04-14	OpenAI	N/A	1M	-
o4-mini-2025-04-16		N/A	200k	✓
DeepSeek-V3-0324	DeepSeek	37B*	64k	-
DeepSeek-R1		37B*	64k	✓
Qwen2.5-Coder-32B	Alibaba	32B	128k	-
QwQ-32B		32B	132k	✓
Llama3.3-70B	Meta	70B	128k	-
Llama4-Maverick		17B*	1M	-

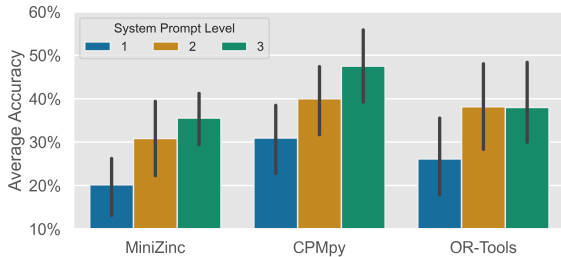


Figure 5: Average Accuracy by Framework and System Prompt Level (Aggregated across LLMs).

CPMpy library for modelling, they exhibit a consistent improvement in performance compared to modelling with MiniZinc and OR-Tools, especially with more detailed system prompts (Levels 2 and 3). The higher accuracy observed with CPMpy is potentially attributable to two factors. First, contemporary LLM training tends to include high quantities of Python code [4]; this familiarity with Python syntax and libraries facilitates generating models in Python-based frameworks. This is also a factor for the increased performance with OR-Tools compared to MiniZinc. Second, CPMpy provides higher-level modelling abstractions compared to OR-Tools, including nested expressions, double reification, as well as a larger set of built-in constraints that can be used. In contrast, the technical low-level interface of OR-Tools often poses challenges for LLMs, leading to reduced accuracy despite the large amount of available documentation and examples.

Looking at the individual performance of LLMs across the frameworks in detail (Figure 4), we can see that the leading models, GPT-

4.1-mini and o4-mini, demonstrate strong capabilities, particularly when generating CPMpy or OR-Tools code, peaking at 63% and 65% respectively. DeepSeek-V3, DeepSeek-R1 and QwQ-32B follow similar trends, although with relatively lower accuracy, achieving their best results with CPMpy (all around ~53% at Level 3). These LLMs benefit from increased prompt detail across the high-level frameworks, but not for the low-level OR-Tools API, where Level 2 prompts (modelling guidelines only) were most effective for DeepSeek-V3 and QwQ-32B. The remaining LLMs generally show lower performance across the frameworks.

Focusing on reasoning LLMs, we observe that o4-mini outperforms all other LLMs on average across frameworks, while QwQ-32B, despite its relatively small size, performs on par with models like GPT-4.1-mini and DeepSeek-V3, and even outperforms both Meta’s Llama models. Interestingly, reasoning models show a relative advantage when modelling with low-level or domain-specific frameworks, as DeepSeek-R1 and o4-mini are the only ones to score above 45% for MiniZinc, and 50% for OR-Tools, while for CPMpy, their non-reasoning counterparts achieved higher accuracies.

In summary, generating models using Python libraries (CPMpy and OR-Tools) tends to yield higher accuracy for most LLMs than the domain-specific MiniZinc language. We note that these results reflect the capabilities of current-generation LLMs; ongoing advances in model architecture and training data might improve accuracy in future iterations.

6.2.2 Q2: System Prompt & Error Types

Figures 4 and 5 also demonstrate the impact of varying system prompt levels across frameworks and LLMs. A key observation across all configurations is that providing at least some specific guidance — either classroom-level guidelines (Level 2) or framework documentation (Level 3) — consistently yields higher accuracy than the basic prompt (Level 1). However, the addition of detailed documentation on top of the guidelines showed some variations. For high-level frameworks like MiniZinc and CPMpy, incorporating API documentation in the system prompts led to an increase in accuracy for 6 out of 8 LLMs. This indicates that more documentation is beneficial to produce valid constraint models when using high-level modelling systems. Interestingly, for the lower-level OR-Tools solver interface, while adding guidelines (Level 2) improved performance over the baseline (Level 1) for all LLMs, further adding detailed documentation (Level 3) did not consistently improve upon Level 2. In fact, average performance slightly decreased, with only 3 out of 8 LLMs showing any benefit from the extra documentation on OR-Tools. This indicates that the LLMs might struggle with integrating extensive

technical details of lower-level modelling frameworks.

To better understand the impact of prompt levels, we examine the types of errors made under different prompt configurations, depicted in Figure 6. We categorize the types of errors into two main categories: detectable and modelling errors. Detectable errors include basic failures like failing to execute the generated code, problems parsing outputs, or running into timeouts. Modelling errors are those instances where either the generated model is unsatisfiable or it produces a wrong solution (according to our solution-level evaluation metric, Section 5.2). For CPMpy, as the prompt level increases, detectable errors decrease while modelling errors rise. A similar trend was observed for MiniZinc and OR-Tools (figures are available in the Appendix). The reduction in detectable errors with more detailed prompts suggests that LLMs generate more syntactically correct and executable code as they receive more information. This partly leads to more successes, but at times also to an increase in logical modelling errors (bottom subfigure), where the code is syntactically correct but the solution is either incorrect or not found.

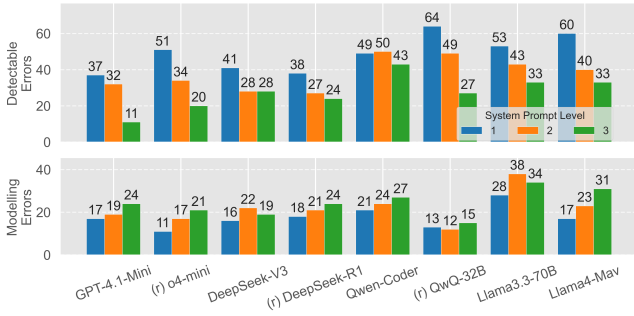


Figure 6: Detectable and modelling errors with CPMpy.

6.2.3 Q3: Inference-Time Compute Methods

Finally, we assess the impact of various inference-time strategies from Section 4, using the following configurations:

1. **Baseline** (Section 4.1): The input consists of the system prompt and problem description. The first generated model is directly evaluated without further processing, as in Q1.
2. **RAICL** (Section 4.2): The input contains the system prompt and $e = 8$ examples based on reverse semantic similarity, following existing work [20]. We use a leave-one-out strategy for evaluation, testing each problem individually while utilizing the 100 remaining ones as the retrieval database.
3. **Sampling** (Section 4.3): The input consists of the system prompt and problem description. The LLM generates $k = 10$ responses, with the final model selected through solution majority voting.
4. **Self-Verification** (Section 4.4): A model is generated with the Baseline method and the LLM iteratively refines it through self-verification, with a maximum number of 10 iterations.
5. **Sampling & Self-Verification**: The Sampling method generates a model, which is input to Self-Verification.

For cost reasons, we included four LLMs (one from each organisation): *gpt-4.1-mini*, *DeepSeek-V3*, *QwQ-32B*, *Llama-3.3-70B*, and we used CPMpy due to its higher accuracy on average (Q1). Based on the Q2 results, we used the documentation system prompt (Level 3). The results are shown in Figure 7.

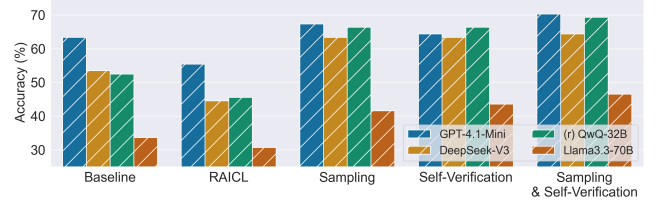


Figure 7: Performance across different inference-time compute methods for each LLM using CPMpy.

Surprisingly, the RAICL strategy was shown ineffective for all tested LLMs; performance consistently decreased compared to the Baseline when in-context examples were provided. Based on these results and compared to the results from Michailidis et al. [20], it seems that LLMs handle more effectively the given structured framework documentation in the system prompt than they generalize from specific problem-model examples provided in context. This simplifies practical deployment by removing the need for custom example database construction and retrieval methods. As such, investigating more modelling tools or frameworks could focus on merely adapting their documentation for the system prompt.

Conversely, the other inference-time compute methods boosted performance across all LLMs, with the most significant gains observed by Qwen’s reasoning LLM QwQ-32B (approx. 20% increase). Repeated sampling improved performance across all LLMs, often showing approximately a 10% increase in accuracy. Notably, with this method, the reasoning-trained QwQ-32B performed on par with DeepSeek’s and OpenAI’s larger LLMs. Self-verification also improved accuracy significantly compared to the Baseline, yielding results comparable to repeated sampling, although gpt-4.1-mini performed slightly better with sampling than with self-verification alone. As RAICL negatively affected accuracy, we experimented with the combination of the best-performing methods: repeated sampling & self-verification. This combined approach further increased accuracy, with all LLMs tested demonstrating their highest performance in this setting.

Overall, the highest accuracy is achieved when prompting gpt-4.1-mini with the documentation prompt and the combined configuration, resulting in 71/101 correct CP models in this 0-shot setting; only 2 instances resulted in detectable errors (1 runtime error, 1 max tokens exceeded), and 28 instances resulted in modelling errors (unsatisfiable or wrong solution).

7 Conclusion

In this work, we introduced a diverse dataset of combinatorial problems and systematically evaluated various state-of-the-art LLMs on transforming textual problem descriptions into executable constraint models. We explored the use of different modelling frameworks, with varying abstraction and interface types. We also explored the use of prompt-based and inference-time compute methods for enhancing the performance of LLMs in constraint modelling.

Our evaluation indicated an advantage of Python-based frameworks (OR-Tools, CPMpy) compared to the domain-specific MiniZinc language, with further improvements when appending detailed guidelines and documentation in system prompts. Reasoning models were shown to be a good fit for this task. Inference-time methods were shown to provide an improvement in general, with repeated sampling and self-verification showing up to a 10% increase in accu-

racy. Notably, retrieval-augmented in-context learning was not effective when combined with increased documentation in the prompts. The strongest performance was obtained from combining repeated sampling and self-verification, achieving 70% accuracy, suggesting that LLMs can indeed assist in the modelling process, but human expert validation is still required, especially for critical tasks.

The existence of CP-Bench allows for systematically evaluating more design choices, including newer LLMs, additional frameworks (e.g., integer linear programming solvers), more coding LLM techniques (e.g. retrieval-augmented documentation), or supervised fine-tuning; the latter would require a training dataset of constraint models and problem descriptions. While our benchmark contains more diverse combinatorial problems, they often originate from textbooks, whereas industrial problems typically involve more data, complex constraints, and elaborate descriptions—though rarely available publicly. Future work could also explore multi-turn interactions, where a user and system iteratively refine the model. Finally, while this paper focused on evaluating model correctness, experimenting with scaled-up problem instances and solving times is worth investigating, with interesting directions being learning to model efficiently or to re-model for efficiency.

Acknowledgements

This project has received funding from the European Research Council (ERC) under the European Union’s Horizon 2020 research and innovation program (Grant No. 101002802, CHAT-Opt)

References

- [1] N. Beldiceanu, M. Carlsson, S. Demasse, and T. Petit. Global constraint catalogue: Past, present and future. *Constraints*, 12:21–62, 2007.
- [2] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.
- [3] D. Chandrasekaran and V. Mago. Evolution of semantic similarity - A survey. *ACM Comput. Surv.*, 54(2):41:1–41:37, feb 2022. ISSN 0360-0300. doi: 10.1145/3440755. URL <https://doi.org/10.1145/3440755>.
- [4] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] X. Chen, M. Lin, N. Schaerli, and D. Zhou. Teaching large language models to self-debug. In *The 61st Annual Meeting Of The Association For Computational Linguistics*, 2023.
- [6] W.-L. Chiang, L. Zheng, Y. Sheng, A. N. Angelopoulos, T. Li, D. Li, H. Zhang, B. Zhu, M. Jordan, J. E. Gonzalez, and I. Stoica. Chatbot arena: An open platform for evaluating llms by human preference, 2024.
- [7] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, and S. H. Tan. Automated repair of programs from large language models. In *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*, pages 1469–1481. IEEE, 2023.
- [8] E. C. Freuder. Progress towards the holy grail. *Constraints An Int. J.*, 23(2):158–171, 2018. doi: 10.1007/s10601-017-9275-0. URL <https://doi.org/10.1007/s10601-017-9275-0>.
- [9] E. C. Freuder and B. O’Sullivan. Grand challenges for constraint programming. *Constraints An Int. J.*, 19(2):150–162, 2014. doi: 10.1007/s10601-013-9155-1. URL <https://doi.org/10.1007/s10601-013-9155-1>.
- [10] A. M. Frisch, C. Jefferson, B. M. Hernández, and I. Miguel. The rules of constraint modelling. In *IJCAI*, pages 109–116, 2005.
- [11] I. P. Gent and T. Walsh. CspLib: A benchmark library for constraints. In J. Jaffar, editor, *Principles and Practice of Constraint Programming – CP’99*, pages 480–481, Berlin, Heidelberg, 1999. Springer Berlin Heidelberg. ISBN 978-3-540-48085-3.
- [12] T. Guns. Increasing modeling language convenience with a universal n-dimensional array, cppy as python-embedded example. In *Proceedings of the 18th workshop on Constraint Modelling and Reformulation at CP (ModRef 2019)*, volume 19, 2019.
- [13] A. Ishay, Z. Yang, and J. Lee. Leveraging large language models to generate answer set programs. In P. Marquis, T. C. Son, and G. Kern-Isberner, editors, *Proceedings of the 20th International Conference on Principles of Knowledge Representation and Reasoning, KR 2023, Rhodes, Greece, September 2-8, 2023, KR ’23*, pages 374–383, 2023. ISBN 978-1-956792-02-7. doi: 10.24963/kr.2023/37. URL <https://doi.org/10.24963/kr.2023/37>.
- [14] E. Jabrayilzade and S. Tekir. Lgpsolver - solving logic grid puzzles automatically. In T. Cohn, Y. He, and Y. Liu, editors, *Findings of the Association for Computational Linguistics: EMNLP 2020, Online Event, 16-20 November 2020*, volume EMNLP 2020 of *Findings of ACL*, pages 1118–1123. Association for Computational Linguistics, 2020. doi: 10.18653/v1/2020.findings-emnlp.100. URL <https://doi.org/10.18653/v1/2020.findings-emnlp.100>.
- [15] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim. A survey on large language models for code generation. *arXiv preprint arXiv:2406.00515*, 2024.
- [16] H. Kjellerstrand. Håkan’s cpmPy examples repository. <http://www.hakank.org/cpmPy/>, 2024. URL <http://www.hakank.org/cpmPy/>. Accessed: 2024-12-24.
- [17] C. Lecoutre and N. Szczepanski. Pycsp3: modeling combinatorial constrained problems in python. *arXiv preprint arXiv:2009.00326*, 2020.
- [18] J. Li, T. Tang, W. X. Zhao, and J. Wen. Pretrained language model for text generation: A survey. In Z. Zhou, editor, *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence, IJCAI 2021, Virtual Event / Montreal, Canada, 19-27 August 2021*, pages 4492–4499. ijcai.org, 2021. doi: 10.24963/ijcai.2021/612. URL <https://doi.org/10.24963/ijcai.2021/612>.
- [19] J. Liu, D. Shen, Y. Zhang, B. Dolan, L. Carin, and W. Chen. What makes good in-context examples for gpt-3? *arXiv preprint arXiv:2101.06804*, 2021.
- [20] K. Michailidis, D. Tsouros, and T. Guns. Constraint Modelling with LLMs Using In-Context Learning. In P. Shaw, editor, *30th International Conference on Principles and Practice of Constraint Programming (CP 2024)*, volume 307 of *Leibniz International Proceedings in Informatics (LIPIcs)*, pages 20:1–20:27, Dagstuhl, Germany, 2024. Schloss Dagstuhl – Leibniz-Zentrum für Informatik. ISBN 978-3-95977-336-2. doi: 10.4230/LIPIcs.CP.2024.20. URL <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.CP.2024.20>.
- [21] K. Michailidis, D. Tsouros, and T. Guns. Cp-bench, June 2025. URL <https://doi.org/10.5281/zenodo.15592407>.
- [22] N. Nethercote, P. J. Stuckey, R. Becket, S. Brand, G. J. Duck, and G. Tack. Minizinc: Towards a standard CP modelling language. In C. Bessière, editor, *Principles and Practice of Constraint Programming - CP 2007, 13th International Conference, CP 2007, Providence, RI, USA, September 23-27, 2007, Proceedings*, volume 4741 of *Lecture Notes in Computer Science*, pages 529–543. Springer, 2007. doi: 10.1007/978-3-540-74970-7_38. URL https://doi.org/10.1007/978-3-540-74970-7_38.
- [23] T. Olausson, A. Gu, B. Lipkin, C. Zhang, A. Solar-Lezama, J. Tenenbaum, and R. Levy. Linc: A neurosymbolic approach for logical reasoning by combining language models with first-order logic provers. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 5153–5176, 2023.
- [24] V. T. Paschos. *Applications of combinatorial optimization*. John Wiley & Sons, 2014.
- [25] L. Perron and F. Didier. Cp-sat, 2024. URL https://developers.google.com/optimization/cp/cp_solver. Accessed: 2025-5-5.
- [26] L. Perron and V. Furnon. Or-tools, 2024. URL <https://developers.google.com/optimization/>. Accessed: 2025-5-5.
- [27] G. Prasath and S. Karande. Synthesis of mathematical programs from natural language specifications. *CoRR*, abs/2304.03287 (arXiv:2304.03287), 2023. doi: 10.48550/arXiv.2304.03287. URL <https://doi.org/10.48550/arXiv.2304.03287>.
- [28] R. Ramamonjison, H. Li, T. T. L. Yu, S. He, V. Rengan, A. Banitalebi-Dehkordi, Z. Zhou, and Y. Zhang. Augmenting operations research with auto-formulation of optimization models from problem descriptions. *CoRR*, abs/2209.15565 (arXiv:2209.15565), 2022. doi: 10.48550/arXiv.2209.15565. URL <https://doi.org/10.48550/arXiv.2209.15565>.
- [29] R. Ramamonjison, T. Yu, R. Li, H. Li, G. Carenini, B. Ghaddar, S. He, M. Mostajabdaveh, A. Banitalebi-Dehkordi, Z. Zhou, et al. Nl4opt competition: Formulating optimization problems based on their natural language descriptions. In *NeurIPS 2022 Competition Track*, pages 189–203. PMLR, 2023.
- [30] H. Simonis. Building industrial applications with constraint programming. In H. Comon, C. Marché, and R. Treinen, editors, *Constraints in Computational Logics: Theory and Applications, International Summer School, CCL’99 Gif-sur-Yvette, France, September 5-8, 1999, Re-*

vised Lectures, volume 2002 of *Lecture Notes in Computer Science*, pages 271–309. Springer, 1999. doi: 10.1007/3-540-45406-3_6. URL https://doi.org/10.1007/3-540-45406-3_6

- [31] A. Singirikonda, S. Kadioglu, and K. Üppuluri. Text2zinc: A cross-domain dataset for modeling optimization and satisfaction problems in minizinc. *arXiv preprint arXiv:2503.10642*, 2025.
- [32] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling llm test-time compute optimally can be more effective than scaling model parameters. *arXiv preprint arXiv:2408.03314*, 2024.
- [33] D. C. Tsouros, H. Verhaeghe, S. Kadioglu, and T. Guns. Holy grail 2.0: From natural language to constraint models. *CoRR*, abs/2308.01589 (arXiv:2308.01589), 2023. doi: 10.48550/arXiv.2308.01589. URL <https://doi.org/10.48550/arXiv.2308.01589>
- [34] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.
- [35] M. Wallace. Practical applications of constraint programming. *Constraints An Int. J.*, 1(1/2):139–168, 1996. doi: 10.1007/BF00143881. URL <https://doi.org/10.1007/BF00143881>
- [36] Y. Weng, M. Zhu, F. Xia, B. Li, S. He, S. Liu, B. Sun, K. Liu, and J. Zhao. Large language models are better reasoners with self-verification. In H. Bouamor, J. Pino, and K. Bali, editors, *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 2550–2575, Singapore, Dec. 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-emnlp.167. URL <https://aclanthology.org/2023.findings-emnlp.167/>
- [37] J. Ye, Z. Wu, J. Feng, T. Yu, and L. Kong. Compositional exemplars for in-context learning. In *International Conference on Machine Learning*, pages 39818–39833. PMLR, 2023.
- [38] X. Ye, Q. Chen, I. Dillig, and G. Durrett. Satlm: Satisfiability-aided language models using declarative prompting. *Advances in Neural Information Processing Systems*, 36:45548–45580, 2023.
- [39] X. Ye, S. Iyer, A. Celikyilmaz, V. Stoyanov, G. Durrett, and R. Pasunuru. Complementary explanations for effective in-context learning. In A. Rogers, J. Boyd-Graber, and N. Okazaki, editors, *Findings of the Association for Computational Linguistics: ACL 2023*, pages 4469–4484, Toronto, Canada, July 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.findings-acl.273.

8 Appendix

Number of Samples

As shown in Figure 8 successes increase for all LLMs as more samples are added in the repeated sampling method.

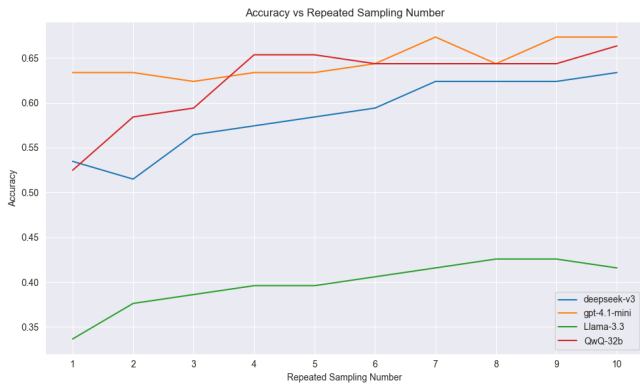


Figure 8: Successes for different number of samples, using repeated sampling with a prespecified system prompt for CPMpy.

Detailed Errors for MiniZinc & OR-Tools

Figures 9 and 10 show the detailed number of detectable and modelling errors per system prompt level and LLM for MiniZinc and OR-Tools respectively.

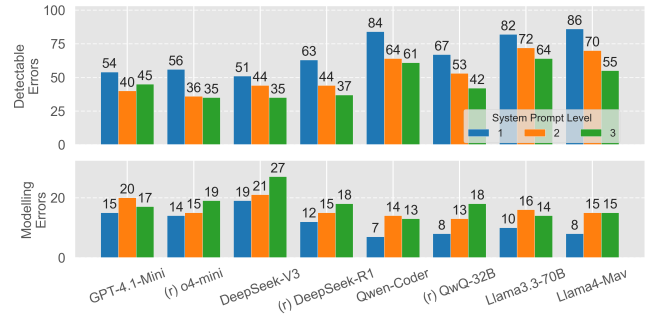


Figure 9: Detectable and modelling errors when LLMs produce MiniZinc code through all system prompt levels.

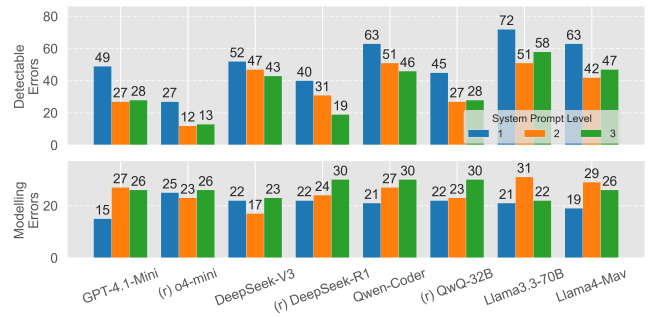


Figure 10: Detectable and modelling errors when LLMs produce OR-Tools code through all system prompt levels.

System Prompts

We used the official documentation platform of each framework to compose the system prompts for our experiments. All system prompts that we created separately for each framework are visible as follows:

- CPMpy^[13] Listings [1](#), [2](#), [4](#)
- MiniZinc^[14] Listings [7](#), [8](#), [9](#)
- OR-Tools^[15] Listings [12](#), [13](#), [15](#)

Self-Debug Prompt

The base self-debug prompt that we utilized for our experiments is presented in Listing [18](#)

An Example Dataset Instance

A dataset instance is shown in Listing [19](#)

An Example LLM Response

Listing [20](#) shows the input composed for an example from our dataset. Listing [21](#) shows the successful initial response from

¹³ Last accessed at July 19, 2024:
<https://cpmpy.readthedocs.io/en/latest/index.html>

¹⁴ Last accessed at July 19, 2024:
<https://docs.minizinc.dev/en/stable/lib-globals.html>

¹⁵ Last accessed at July 19, 2024:
<https://developers.google.com/optimization/cp>

Listing 1 Basic Prompt (Level 1) for CPMpy.

You are a coding assistant with expertise in Python, constraint programming, and modelling combinatorial problems. Your task is to create Python code using the CPMpy library to solve constraint problems. Only the standard Python libraries and CPMpy should be used.

Output Formatting

Here is an example for printing, if the problem description contains: “Print the number of apples and oranges (apples, oranges), the cost per round (cost), and the total cost (total_cost).”. In this case, the output of the solution as JSON should be done as follows:

```
“python
if model.solve():
    solution = {'apples': int(apples.value()),
               'oranges': int(oranges.value()), 'cost':
               cost.value().tolist(), 'total_cost':
               int(model.objective_value())} #
    assuming 'model' is the CPMpy model,
    and 'apples', 'oranges', 'cost' are the
    decision variables, and the objective
    has been set to minimize or maximize
    the total cost
    print(json.dumps(solution, indent=4))
else:
    print("No solution found.")
“
```

deepseek-coder to this input. Then, the model is given to a solver which gives a solution; the self-debug prompt is given as in Listing 22 and the final response of the LLM is given in Listing 23, agreeing to its correctness.

Listing 2 Guidelines Prompt (Level 2) for CPMpy (Part A).

[[Basic Prompt]]

Guidelines

Code Generation Steps

1. Import only the necessary libraries (cpmpy, json, numpy, etc.). Avoid the use of other libraries except for standard Python libraries, CPMpy, and numpy.
2. Extract and process the provided data, if any.
3. Define decision variables.
4. Construct a model with appropriate constraints.
5. Set the objective, if applicable.
6. Solve the model.
7. Print the solution in JSON format according to the print statement. The print statement is the last part of the problem description. The keys should be exactly the same as the decision variables given in the parentheses of the print statement. The values should be integers or lists of integers, or lists of lists of integers, etc.

Mandatory guidelines:

- Be clear, logically organized, and include comments explaining each step. Keep your code simple and maintainable to prevent syntax errors.
- Use .value().tolist() method to get solution values from array variables (of any dimension).
- When printing integer values (e.g objective values, integer variable values, sums, etc.), always wrap them with int() to avoid JSON serialization issues.
- Avoid deprecated functions (e.g., use intvar instead of IntVar).
- Do not use fractional or decimal numbers as parameters, only integers.
- Be careful when using the built-in functions sum, max, min, all, and any. They are overloaded in CPMpy. If you write ‘from cpmpy import *’, then the overloaded functions will be used instead of the Python built-in functions. If you write ‘import cpmpy as cp’, then you can use the CPMpy-overloaded built-in functions by calling cp.sum, cp.max, cp.min, cp.all, and cp.any.
- Consider edge cases and possible errors that may occur during the execution of the code.

[[continued at Listing 3]]

Listing 3 Guidelines Prompt (Level 2) for CPMpy (Part B).

[[continuing from Listing 2]]

Response Format

You must generate self-contained and self-runable code, between ‘`python`’ and ‘`python`’ tags. Nothing else should be included in the response, only the code. So, the response should look like this:

```
python
import cpmPy as cp
import json

# Data (optional)
...
# End of data

# Model definition
model = cp.Model()

# Decision Variables
...

# Constraints
model += ...

# Objective (optional)
model.minimize(objective) # or model.maximize(
    objective)

# Solve and print
if model.solve():
    solution = {...}
    print(json.dumps(solution, indent=4))
else:
    print("No solution found.")
python
```

Important:

The generated code should always print the solution in JSON format using as keys the decision variables as given in the parentheses in the problem description’s print statement. The only values allowed are integers and lists of (lists of ...) integers. If booleans are requested, use 0 for False and 1 for True. This is really important for the evaluation of your response, because I will directly assign these values to the variables of the ground-truth model and check if the constraints (and objective) are satisfied, so follow these guidelines carefully.

Listing 4 Documentation Prompt (Level 3) for CPMpy (Part A).

[[Guidelines Prompt]]

CPMpy Documentation

CPMpy is a Constraint Programming and Modeling library in Python, based on numpy, with direct solver access.

An example of a CPMpy model:

```
python
import cpmPy as cp
import json

model = cp.Model()

# Variables
b = cp.boolvar(name="b")
x1, x2, x3 = x = cp.intvar(1,10, shape=3, name="x") # unpacking

# Constraints
model += (x[0] == 1)
model += cp.AllDifferent(x)
model += b.implies(x[1] + x[2] > 5)

# Objective function (optional)
model.maximize(cp.sum(x) + 100*b)

if model.solve():
    solution = {'b': int(b.value()), 'x': x.value().tolist()}
    print(json.dumps(solution, indent=4))
else:
    print("No solution found.")
python
```

Short API Documentation:

- class `cpmPy.model.Model(*args, minimize=None, maximize=None)`: CPMpy Model object, contains the constraint and objective expressions. The methods of the Model class are:
 - `maximize(expr)`: Maximize the given objective function
 - `minimize(expr)`: Minimize the given objective function
 - `objective_value()`: Returns the value of the objective function of the latest solver run on this model
 - `solve(solver=None, time_limit=None)`: Send the model to a solver and get the result (true if a solution was found), solver -> default: None = first available solver

[[continued at Listing 5]]

Listing 5 Documentation Prompt (Level 3) for CPMpy (Part B).

[[continuing from Listing 4]]

- Global Constraints (`cpmpy.expressions.globalconstraints`):
 - `AllDifferent(*args)`: All arguments have a different (distinct) value
 - `AllDifferentExcept0(*args)`: All nonzero arguments have a distinct value
 - `AllEqual(*args)`: All arguments have the same value
 - `Circuit(*args)`: The sequence of variables form a circuit, where $x[i] = j$ means that j is the successor of i .
 - `Cumulative(start, duration, end, demand, capacity)`: Global cumulative constraint. Used for resource aware scheduling. Ensures no overlap between tasks and never exceeding the capacity of the resource Supports both varying demand across tasks or equal demand for all jobs
 - `GlobalCardinalityCount(vars, vals, occ)`: The number of occurrences of each value `vals[i]` in the list of variables `vars` must be equal to `occ[i]`.
 - `IfThenElse(condition, if_true, if_false)`: Conditional expression that returns `if_true` if condition is true, otherwise returns `if_false`.
 - `Table(array, table)`: The values of the variables in 'array' correspond to a row in 'table'
 - `Xor(arg_list)`: The 'xor' exclusive-or constraint
 - ... <the rest of the constraints are omitted for brevity>
- Global Functions (`cpmpy.expressions.globalfunctions`):
 - `Minimum(arg_list)`: Computes the minimum value of the arguments
 - `Maximum(arg_list)`: Computes the maximum value of the arguments
 - `Abs(expr)`: Computes the absolute value of the expression
 - `Element(arr, idx)`: The 'Element' global constraint enforces that the result equals `Arr[Idx]` with 'Arr' an array of constants or variables and 'Idx' an integer decision variable, representing the index into the array.
 - `Count(arr, val)`: Represents the number of occurrences of `val` in `arr`
 - `Among(arr, vals)`: Represents the number of variables that take values among the values in `vals`
 - `NValue(arr)`: Counts the number of distinct values in a given set of variables
 - `NValueExcept(arr, n)`: Counts the number of distinct values, not including value `n`, if any argument is assigned to it.

[[continued at Listing 6]]

Listing 6 Documentation Prompt (Level 3) for CPMpy (Part C).

[[continuing from Listing 5]]

- Python builtin overwrites (`cpmpy.expressions.python_builtins`):
 - `max(iterable)`: if the iterable contains a CPMpy expression, it posts a `cp.Maximum()` global function, otherwise identical to numpy's `max()`
 - `min(iterable)`: as above, but with `cp.Minimum()` and numpy's `min()`
 - `all(iterable)`: if the iterable contains a CPMpy expression, then it posts a listwise 'and' operator, otherwise identical to numpy's `all()`
 - `any(iterable)`: as above, but with listwise 'or' operator and numpy's `any()`
 - `sum(iterable)`: if the iterable contains a CPMpy expression, then it posts a sum operator, otherwise identical to numpy's `sum()`
- Core expressions (`cpmpy.expressions.core`):
 - Comparisons:
 - * `x == y`
 - * `x != y`
 - * `x < y`
 - * `x <= y`
 - * `x > y`
 - * `x >= y`
 - Mathematical operators:
 - * `-x`
 - * `x + y`
 - * `sum([x,y,z])`
 - * `sum([c0*x, c1*y, c2*z])`
 - * `x - y`
 - * `x * y`
 - * `x // y` (integer division only; never use float division)
 - * `x % y`
 - Logical operators:
 - * `x & y` (and)
 - * `x | y` (or)
 - * `~ x` (not)
 - * `x^y` (xor)
 - Python has no built-in operator for `__implication__` that can be overloaded. CPMpy hence has a function 'implies()' for decision variables, which can be used: `x.implies(y)`

Listing 7 Basic Prompt (Level 1) for MiniZinc.

You are a coding assistant with expertise in MiniZinc, constraint programming, and modelling combinatorial problems. Your task is to create MiniZinc code to solve constraint problems. Only the standard MiniZinc library and global constraints should be used (e.g., `globals.mzn`).

Output Formatting

Here is an example for printing, if the problem description contains: “Print the number of apples and oranges (apples, oranges), the cost per round (cost), and the total cost (total_cost).”. In this case, the output of the solution as JSON should be done as follows:

```
“minizinc
  output [
    "{",
    join(",", [
      "\"" ++ "apples" ++ "\": " ++ show(
        apples),
      "\"" ++ "oranges" ++ "\": " ++ show(
        oranges),
      "\"" ++ "cost" ++ "\": [" ++ join(",",
        [show(apple_cost[r]) | r in 1..
        num_rounds]) ++ "]",
      "\"" ++ "total_cost" ++ "\": " ++ show(
        total_value)
    ]),
    "]"
  ];
“
```

Listing 8 Guidelines Prompt (Level 2) for MiniZinc.

[[Basic Prompt]]

Guidelines

Code Generation Steps

1. Import necessary libraries (globals, etc.).
2. Extract and process the provided data.
3. Define decision variables.
4. Construct a model with appropriate constraints.
5. Solve the model (satisfy, minimize, maximize).
6. Print the solution in JSON format using as keys exactly the same variable names as given in the problem description’s parentheses (at the print description sentence, which is normally at the end of the problem description). The values should be the decision variables’ values or the objective value.

Mandatory guidelines:

- Be clear, logically organized, and include comments explaining each step.
- Use ‘show’ function to get solution values from variables.
- Avoid deprecated functions.
- Keep your code simple and maintainable to prevent syntax errors.
- Consider edge cases and possible errors that may occur during the execution of the code.

Response Format

You should generate self-contained and self-runable code, between “minizinc and “ tags. Nothing else should be included in the response, only the code. So, the response should look like this:

```
“minizinc
  % Include libraries if needed
  include ...
  % Input data
  ...
  % Decision Variables
  ...
  % Constraints
  ...
  solve satisfy;
  % or solve minimize or solve maximize
  output [{"..."}];
“
```

Important:

The generated code should always print the solution in JSON format using as keys the decision variables as given in the parentheses in the problem description, and the only values allowed are integers (or booleans) and lists of (lists of ...) integers (or booleans). This is really important for the evaluation of your response, because I will directly assign these values to the variables of the ground-truth model and check if the constraints (and objective) are satisfied, so follow these guidelines carefully.

Listing 9 Documentation Prompt (Level 3) for MiniZinc (Part A).

[[Guidelines Prompt]]

MiniZinc Documentation

Minizinc is a high-level constraint modeling language that allows you to define and solve constraint satisfaction and optimization problems.

An example of a Minizinc model:

```
“minizinc
    include "globals.mzn";

    % Variables
    var bool: b;
    array[1..3] of var 1..10: x;

    % Constraints
    constraint x[1] = 1;
    constraint alldifferent(x);
    constraint b -> (x[2] + x[3] > 5);

    % Objective function (optional)
    solve maximize sum(x) + 100 * bool2int(b);

    output [
        "{\n",
        "    \"b\": ", show(bool2int(b)), ", \"n\",
        "    \"x\": ", show(x), \"\n\",
        "  }"
    ];
...

```

Short Documentation:

- Global Constraints, Predicates and Standard Library Functions:
 - predicate `all_different(array [$X] of var int: x)`: Constrain the elements in the array `x` to be pairwise different.
 - predicate `all_different_except_0(array [$X] of var int: vs)`: Constrain the elements of the array of integers `vs` to be pairwise different except for those elements that are assigned the value 0.
 - predicate `all_equal(array [$X] of var int: x)`: Constrain the elements of the array `x` to be all equal.
 - predicate `all_disjoint(array [$X] of var set of int: S)`: Constrain the array of sets of integers `S` to be pairwise disjoint.
 - predicate `circuit(array [$E] of var $E: x)`: Constrains the elements of `x` to define a circuit where `x[i] = j` means that `j` is the successor of `i`.
 - predicate `cumulative(array [int] of var int: s, array [int] of var int: d, array [int] of var int: r, var int: b)`: Requires that a set of tasks given by start times `s`, durations `d`, and resource requirements `r`, never require more than a global resource bound `b` at any one time. Assumptions: forall `i`, $d[i] \geq 0$ and $r[i] \geq 0$.

[[continued at Listing 10]]

Listing 10 Documentation Prompt (Level 3) for MiniZinc (Part B).

[[continuing from Listing 9]]

• Cont.

- predicate `global_cardinality(array [$X] of var $E: x, array [$Y] of $E: cover, array [$Y] of var int: counts)`: Requires that the number of occurrences of `cover[i]` in `x` is `counts[i]`.
- predicate `if_then_else(array [int] of var bool: c, array [int] of int: x, var int: y)`: Conditional constraint. This constraint is generated by the compiler for if-then-else expressions. The last entry in the `c` array is always the constant `true`, corresponding to the else case.
- predicate `table(array [$E] of var bool: x, array [int,$E] of bool: t)`: Represents the constraint `x` in `t` where we consider each row in `t` to be a tuple and `t` as a set of tuples.
- predicate `'xor'(var bool: x, var bool: y)`: Return truth value of `x xor y`. Usage: `x xor y`
- predicate `minimum(var float: m, array [int] of var float: x)`: Constrains `m` to be the minimum of the values in `x`. Assumptions: $|x| > 0$.
- predicate `maximum(var $E: m, array [int] of var $E: x)`: Constrains `m` to be the maximum of the values in `x`. Assumptions: $|x| > 0$.
- function `int: abs(int: x)`: Computes the absolute value of the expression.
- predicate `element(var $E: i, array [$E] of var bool: x, var bool: y)`: Constrains `i` to be the index of the element `y` in the array `x`.
- predicate `member(array [int] of var bool: x, var bool: y)`: Requires that `y` occurs in the array `x`.
- predicate `count(array [$X] of var opt $E: x, var $E: y, var int: c)`: Constrains `c` to be the number of occurrences of `y` in `x`.
- predicate `among(var int: n, array [$X] of var $E: x, set of $E: v)`: Requires exactly `n` variables in `x` to take one of the values in `v`.
- predicate `nvalue(var int: n, array [$X] of var int: x)`: Requires that the number of distinct values in `x` is `n`.
- predicate `increasing(array [$X] of var bool: x)`: Requires that the array `x` is in increasing order (duplicates are allowed).
- predicate `inverse(array [$X] of var $Y: f, array [$Y] of var $X: invf)`: Constrains two arrays of int variables, `f` and `invf`, to represent inverse functions. All the values in each array must be within the index set of the other array.
- predicate `at_least(int: n, array [$X] of var set of $E: x, set of $E: v)`: Requires at least `n` variables in `x` to take the value `v`.

[[continued at Listing 11]]

Listing 11 Documentation Prompt (Level 3) for MiniZinc (Part C).

[[continuing from Listing 10]]

- Cont.
 - predicate `exactly(int: n, array [$X] of var set of $E: x, set of $E: v)`: Requires exactly n variables in x to take the value v .
 - predicate `disjoint(var set of $E: s1, var set of $E: s2)`: Requires that sets $s1$ and $s2$ do not intersect.
 - function `var $E: arg_max(array [$E] of var int: x)`: Returns the index of the maximum value in the array x . When breaking ties the least index is returned.
 - predicate `range(array [$X] of var $Y: x, var set of $X: s, var set of $Y: t)`: Requires that the image of function x (represented as an array) on set of values s is t . $ub(s)$ must be a subset of $index_set(x)$ otherwise an assertion failure will occur.
 - predicate `subcircuit(array [$E] of var $E: x)`: Constrains the elements of x to define a subcircuit where $x[i] = j$ means that j is the successor of i and $x[i] = i$ means that i is not in the circuit.
 - predicate `sum_set(array [$X] of $Y: vs, array [$X] of int: ws, var set of $Y: x, var int: s)`: Requires that the sum of the weights $ws[i1]..ws[iN]$ equals s , where $vs[i1]..vs[iN]$ are the elements appearing in set x .
- Core Expressions:
 - Comparisons:
 - * $x == y$
 - * $x! = y$
 - * $x < y$
 - * $x \leq y$
 - * $x > y$
 - * $x \geq y$
 - Mathematical operators:
 - * $-x$
 - * $x + y, x - y$
 - * $sum([x, y, z])$
 - * $sum([c0 * x, c1 * y, c2 * z])$
 - * $x * y, x / y$
 - * $x \bmod y$
 - Logical operators:
 - * $x \wedge y$ (and)
 - * $x \vee y$ (or)
 - * $\text{not } x$ (not)
 - * $x \text{ xor } y$ (exclusive or)
 - * $x \rightarrow y$ (implication)

Solving:

solve satisfy: Find any solution that satisfies all constraints.

solve minimize expr: Find a solution that minimizes 'expr'.

solve maximize expr: Find a solution that maximizes 'expr'.

Listing 12 Basic Prompt (Level 1) for OR-Tools.

You are a coding assistant with expertise in Python, constraint programming, and modelling combinatorial problems. Your task is to create Python code using the OR-tools library to solve constraint problems. Only the standard Python libraries and OR-tools should be used.

Output Formatting

Here is an example for printing, if the problem description contains: "Print the number of apples and oranges (apples, oranges), the cost per round (cost), and the total cost (total_cost)". In this case, the output of the solution as JSON should be done as follows:

```
“python
    if status == cp_model.OPTIMAL or status ==
        cp_model.FEASIBLE:
        solution = {
            'apples': solver.Value(apples),
            'oranges': solver.Value(oranges),
            'cost': [solver.Value(c) for c in cost
                ],
            'total_cost': solver.ObjectiveValue()
        }
        print(json.dumps(solution, indent=4))
    else:
        print("No solution found.")
“
```

Listing 13 Guidelines Prompt (Level 2) for OR-Tools (Part A).

[[Basic Prompt]]

Guidelines

Code Generation Steps

1. Import necessary libraries (ortools.sat.python.cp_model, json, etc.).
2. Extract and process the provided data.
3. Define decision variables.
4. Construct a model with appropriate constraints.
5. Set the objective, if applicable.
6. Solve the model.
7. Print the solution in JSON format using as keys exactly the same variable names as given in the problem description's parentheses (at the print description sentence, which is normally at the end of the problem description). The values should be the decision variables' values or the objective value.

Mandatory guidelines:

- Be clear, logically organized, and include comments explaining each step.
- Use `.Value()` method to get solution values from variables.
- Avoid deprecated functions.
- Keep your code simple and maintainable to prevent syntax errors.
- Consider edge cases and possible errors that may occur during the execution of the code.

[[continued at Listing 14]]

Listing 14 Guidelines Prompt (Level 2) for OR-Tools (Part B).

[[continuing from Listing 13]]

Response Format

You should generate self-contained and self-runnable code, between “python and ” tags. Nothing else should be included in the response, only the code. So, the response should look like this:

```
“python
from ortools.sat.python import cp_model
import json

# Input data
...

# Model definition
model = cp_model.CpModel()

# Decision Variables
...

# Constraints
...

# Objective function (if any)
model.Minimize(objective)
# or model.Maximize(objective)

# Solve the model
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.OPTIMAL or status ==
    cp_model.FEASIBLE:
    solution = {...}
    print(json.dumps(solution, indent=4))
else:
    print("No solution found.")
“
```

Important:

The generated code should always print the solution in JSON format using as keys the decision variables as given in the parentheses in the problem description, and the only values allowed are integers (or booleans) and lists of (lists of ...) integers (or booleans). This is really important for the evaluation of your response, because I will directly assign these values to the variables of the ground-truth model and check if the constraints (and objective) are satisfied, so follow these guidelines carefully.

Listing 15 Documentation Prompt (Level 3) for OR-Tools (Part A).

[[Guidelines Prompt]]

OR-Tools Documentation

OR-tools is an open-source software suite for optimization, including constraint programming, linear programming, and mixed-integer programming.

An example of an OR-tools model:

```
“python
from ortools.sat.python import cp_model
import json

# Model definition
model = cp_model.CpModel()

# Variables
b = model.NewBoolVar('b')
x = [model.NewIntVar(1, 10, f'x{i}') for i in
     range(3)]

# Constraints
model.Add(x[0] == 1)
model.AddAllDifferent(x)
model.Add(x[1] + x[2] > 5).OnlyEnforceIf(b)

# Objective function (optional)
model.Maximize(sum(x) + 100 * b)

# Solve the model
solver = cp_model.CpSolver()
status = solver.Solve(model)

if status == cp_model.OPTIMAL or status ==
    cp_model.FEASIBLE:
    solution = {'b': solver.Value(b), 'x': [
        solver.Value(var) for var in x]}
    print(json.dumps(solution, indent=4))
else:
    print("No solution found.")
“
```

Short Documentation:

- **Model:**
 - ‘model = cp_model.CpModel()’: Create a new model.
- **Variables:**
 - ‘model.NewIntVar(lb, ub, 'name')’: Create a new integer variable with lower bound ‘lb’ and upper bound ‘ub’.
 - ‘model.NewBoolVar('name')’: Create a new boolean variable.
 - ‘model.NewIntervalVar(start, size, end, 'name')’: Create a new interval variable with start, size, and end. An interval variable is a constraint, that is itself used in other constraints like NoOverlap. Internally, it ensures that ‘start + size == end’.

[[continued at Listing 16]]

Listing 16 Documentation Prompt (Level 3) for OR-Tools (Part B).
[[continuing from Listing 15]]

- Constraints:
 - ‘model.Add(expr)’: Add a constraint to the model.
 - ‘model.AddAllDifferent(*expressions)’: This constraint forces all expressions to have different values.
 - ‘model.AddCircuit(arcs: Sequence[ArcT])’: Adds a circuit constraint from a sparse list of arcs that encode the graph.
 - ‘model.AddCumulative(interval_vars, demands, capacity)’: The cumulative constraint. Each interval in ‘interval_vars’ requires ‘demands[i]’ resources and the capacity is ‘capacity’.
 - ‘model.AddElement(index, variables, target)’: Enforces ‘variables[index] == target’.
 - ‘model.AddImplication(a, b)’: Adds the implication constraint ‘a $\rightarrow$ b’.
 - ‘model.AddAllowedAssignments(variables, tuple_list)’: An AllowedAssignments constraint is a constraint on an array of variables, which requires that when all variables are assigned values, the resulting array equals one of the tuples in ‘tuple_list’.
 - ‘model.AddAbsEquality(target, expr)’: Adds ‘target == Abs(expr)’.
 - ‘model.AddModuloEquality(target, expr, mod)’: Adds ‘target = expr % mod’.
 - ‘model.AddBoolXOr(literals)’: Adds ‘XOr(literals) == true’.
 - ‘model.AddBoolOr(literals)’: Adds ‘Or(literals) == true’.
 - ‘model.AddBoolAnd(literals)’: Adds ‘And(literals) == true’.
 - ‘model.AddAtLeastOne(literals)’: Same as ‘add_bool_or’: ‘sum(literals) $\geq$ 1’.
 - ‘model.AddAtMostOne(literals)’: Adds ‘AtMostOne(literals)’: ‘sum(literals) $\leq$ 1’.
 - ‘model.AddExactlyOne(literals)’: Adds ‘ExactlyOne(literals)’: ‘sum(literals) == 1’.
 - ‘model.AddNoOverlap2D(x_intervals, y_intervals)’: Adds a 2D no-overlap constraint.
 - ‘model.AddNoOverlap(intervals)’: Adds a no-overlap constraint.

[[continued at Listing 17]]

Listing 17 Documentation Prompt (Level 3) for OR-Tools (Part C).
[[continuing from Listing 16]]

- Reification:
 - ‘constraint.OnlyEnforceIf(boolvar)’: Adds an enforcement literal to the constraint. The constraint is only enforced if the enforcement literal is true.
 - Objective Function:
 - ‘model.Maximize(expr)’: Define an expression to maximize.
 - ‘model.Minimize(expr)’: Define an expression to minimize.
 - Solver:
 - ‘solver = cp_model.CpSolver()’: Create a solver.
 - ‘solver.Solve(model)’: Solve the model.
 - ‘solver.Value(var)’: Get the value of the variable ‘var’ in the solution.
 - ‘solver.ObjectiveValue()’: Get the value of the objective function.
 - Solving:
 - ‘solver.Solve(model)’: Find a solution that satisfies all constraints.
 - ‘solver.SolveWithSolutionCallback(model, callback)’: Solve the model and call the callback at each solution found.
 - ‘solver.SearchForAllSolutions(model, callback)’: Search for all solutions and call the callback for each one.
-

Listing 18 Base Self-Debug Prompt

You will be given a combinatorial problem description with instructions on how to model it, its code formulation according to these instructions, and the code output. Explain the given code line by line, especially elaborating on the decision variables, constraints, and the objective (if applicable). Then, evaluate the code’s correctness in three aspects:

1. **Runtime:** Does the code run successfully without syntax errors, and does it correctly utilize the required libraries?
2. **Model:** Are the decision variables, constraints, and objective (if applicable) correctly defined? Does the generated solution satisfy the constraints and objective of the given problem description?
3. **Solution Printing:** Does the code print the solution in the required format according to the given instructions?

If the code is correct, end your response with [[OK]]. If the code is incorrect, provide a corrected version of the code between “python and “”, ensuring the fixed code is self-contained and runnable. End your response with [[FIXED]].

Note: Use [[OK]] and [[FIXED]] only once at the end of your response, and only one of them.

Listing 19 An example instance from the dataset: Clock Triplets Problem

```
# Source: http://www.hakank.org/cpmpy/
clock_triplets.py
# Source description: http://www.flcompiler.com
/samples/Dean%20Clark%27s%20Problem.fl.html

"""Rearrange the numbers on the face of a
clock (1 to 12) so no triplet of
adjacent numbers has a sum higher than
21. This is the smallest value that the
highest sum of a triplet can have. Print
the arrangement of the 12 numbers on the
clock (x) as a list, with the first
number being 12."""

from cpmpy import *
import json

# Parameters
n = 12

# Decision Variables
x = intvar(1, n, shape=n)
triplet_sum = intvar(0, 21)

# Constraints
model = Model()
model += AllDifferent(x)
model += x[0] == 12
model += x[1] > x[11]
model += [(x[i % 12] + x[(i % 12) - 1] + x[(i % 12) - 2]) <= triplet_sum for i in range(n)]

# Solve
model.solve()

# Print the solution
solution = {"x": x.value().tolist()}
print(json.dumps(solution))
# Output: {"x": [12, 5, 4, 10, 7, 2, 9, 8, 1, 11, 6, 3]}
```

Listing 20 An example input prompt for a problem from the dataset
[[System Prompt]]

The problem that you have to model is the following: Given an integer $n > 0$ and an integer $m \geq 0$, find a non-empty finite series $S=(s_0, s_1, \dots, s_n, s_{n+1})$ such that (1) there are s_i occurrences of i in S for each integer i ranging from 0 to n , and (2) $s_{n+1}=m$. Print the series S (s).

Input data: $n = 27$ $m = 5$ ### End of input data

Listing 21 The initial response from deepseek-coder to the input of Listing 20

```
```python
import cpmpy as cp
import json

Input data
n = 27
m = 5

Model definition
model = cp.Model()

Decision Variables
s = cp.intvar(0, n+1, shape=n+2, name="s")

Constraints
Constraint (1): there are si occurrences of i
in S for each integer i ranging from 0 to
n
for i in range(n+1):
 model += [cp.sum([s[j] == i for j in range(
 n+2)]) == s[i]]

Constraint (2): sn+1=m
model += [s[n+1] == m]

Solve and print
if model.solve():
 solution = {'s': s.value().tolist()}
 print(json.dumps(solution, indent=4))
else:
 print("No solution found.")
```
```

Listing 22 The self-debug prompt given to the LLM, after its response as in Listing 21

You will be given a combinatorial problem description with instructions on how to model it, its code formulation according to these instructions, and the code output. Explain the given code line by line, especially elaborating on the decision variables, constraints, and the objective function (if applicable). Then, evaluate the code's correctness in three aspects:

- Runtime**: Does the code run successfully without syntax errors, and does it correctly utilize the required libraries?
- Model**: * Are the decision variables, constraints, and objective function (if applicable) correctly defined? * Does the generated solution satisfy the constraints and objective of the given problem description?
- Solution printing**: Does the code print the solution in the required JSON format, with the correct keys and values according to the given instructions?

If the code is correct, end your response with `[[OK]]`. If the code is incorrect, provide a corrected version of the code between ````python` and `````, ensuring the fixed code is self-contained and runnable. End your response with `[[FIXED]]`. Note: Use `[[OK]]` and `[[FIXED]]` only once at the end of your response, and only one of them.

Here are the instructions of the user (between ````instr` and ````` tags):
````instr [[System Prompt]] ``` Here is the given problem description (between ```desc and ``` tags): ```desc [[Problem Description]] ``` Here is the generated code (between ```python and ```): ```python [[Initial Model]] ``` Here is the output of the generated code (between ```out and ```): ```out "s": [23,2,2,...,5] ````

**Listing 23** The self-debug verification response from the LLM, after the prompt as in Listing 22 (Part A).

```

Let's analyze the given code line by
line, focusing on the decision variables,
constraints, and the objective function
(if applicable).
'''python
import cpmpy as cp
import json

Input data
n = 27
m = 5

Model definition
model = cp.Model()

Decision Variables
s = cp.intvar(0,n+1,shape=n+2,name="s")
'''
- **Decision Variables**: 's' is defined as an
array of 'n+2' integer variables, each
ranging from '0' to 'n+1'. This array
represents the series 'S' with 'n+2'
elements.

'''python
Constraints
Constraint (1): there are si occurrences of i
in S for each integer i ranging from 0 to
n
for i in range(n+1):
 model += [cp.sum([s[j] == i for j in range(
n+2)]) == s[i]]
'''
- **Constraint (1)**: This loop ensures that
for each integer 'i' from '0' to 'n', the
number of occurrences of 'i' in the series
'S' (i.e., the number of 'j' such that 's[j]
== i') is equal to 's[i]'.

'''python
Constraint (2): sn+1=m
model += [s[n+1] == m]
'''
- **Constraint (2)**: This ensures that the
last element of the series 'S', 's[n+1]',
is equal to 'm'.

'''python
Solve and print
if model.solve():
 solution = {'s': s.value().tolist()}
 print(json.dumps(solution, indent=4))
else:
 print("No solution found.")
'''
- **Solve and Print**: The model is solved, and
if a solution is found, it is printed in
JSON format with the key 's' and the value
being the list of integers representing the
series S.

```

[[continued at Listing 24]]

**Listing 24** The self-debug verification response from the LLM, after the prompt as in Listing 22 (Part B).

[[continuing from Listing 23]]

```

Evaluation of Code Correctness
a) **Code Correctness**:
- The code runs successfully without syntax
errors.
- It correctly utilizes the required libraries
('cpmpy', 'json').

b) **Model Correctness**:
- **Decision Variables**: Correctly defined as
an array of integers.
- **Constraints**:
- Constraint (1) correctly ensures the
number of occurrences of each integer '
i' in the series 'S'.
- Constraint (2) correctly ensures the last
element of the series 'S' is equal to
'm'.
- **Objective Function**: Not applicable in
this problem.
- The generated solution satisfies the
constraints of the given problem
description.

c) **Solution Printing**:
- The code prints the solution in the required
JSON format with the correct key 's' and
the values are integers.

```

Given the analysis, the code is correct in all aspects.

[[OK]]